EvenUp - Software Requirements Specification

A Web-Based Shared Expense and Bill-Splitting System

SEM V

Prepared by: Pratiksha Rajesh Gaikwad

Roll No: 66010

Guide: Mrs Bhakti Raut

Department of Computer Science

Institution:

Sonopant Dandekar Shikshan Mandali's

Sonopant Dandekar Arts, V.S. Apte Commerce

and M.H. Mehta Science College, Palghar

Academic year: 2026–2027

Table of Contents

Section Title	Page No.
Abstract	**1**
1. Introduction	**2-3**
2. Problem Statement	**4-5**
3. Feasibility Study	**6-7**
4. Requirement Engineering	**8-15**
5. Software Development Life Cycle Planning	**16**
6. System Modelling Using UML	**17-22**
7. System Architecture Design	**23-25**
8. Implementation, Testing, Deployment, and Evaluation	**26-33**
9. Conclusion and Future Scope	**34**
10. References	**35**

Abstract

Shared expenses are common in households, travel groups, student teams, social events, and small institutions. The usual approach is to depend on mental arithmetic, chat messages, paper notes, or a spreadsheet maintained by one person. These approaches become unreliable when different people pay for different items, when an expense is shared by only some members, or when a previous debt is settled later. The result is often an unclear answer to three practical questions: who paid, who owes, and who should pay whom.

EvenUp is a web-based shared-expense management and bill-splitting system designed to make those answers explicit. The system provides account registration and login, optional Google OAuth integration, profile management, group creation, group membership, expense recording, equal or selected-member splitting, running balances, settlement suggestions, settlement recording, group communication, expense history, interactive insights, and report export controls. It stores the expense and settlement records in a MySQL database and presents the system through a responsive Flask and Jinja web interface.

This document specifies the problem, feasibility, requirements, development plan, nine UML models, architecture, implementation status, testing strategy, deployment plan, performance evidence, and evaluation criteria. The current project is an academic prototype under active development. The recorded performance test is a local development baseline and must not be interpreted as a production capacity guarantee.

1. Introduction

1.1 Background

A shared expense is an expenditure made by one person for the benefit of a group or a selected part of that group. Examples include groceries purchased by a flatmate, hotel costs paid by one traveller, meals paid by a friend, or event materials purchased by a student committee. A correct split requires more than dividing a total by the number of people. The payer, the participating members, the currency, the category, the date, and previous settlements all affect the final balance.

EvenUp addresses this problem with a central, shared record. Each expense is connected to a group, a payer, an amount, a category, and one or more expense shares. The application uses these records to calculate what each member has paid, what each member should bear, and the resulting amount to receive or pay. Settlement suggestions are then produced from the balances so that members can resolve the group debt with fewer, clearer transactions.

1.2 Purpose of the SRS

The purpose of this document is to define the functional and non-functional requirements of EvenUp in a form that can guide implementation, testing, evaluation, and future maintenance. It also records the actual state of the academic project so that completed, partially completed, and planned work are distinguishable.

The SRS serves as a common reference for the student developer, project guide, evaluator, and future contributors. It explains why the system is required, what the system must do, how its components interact, how it will be tested, and how it can be deployed.

1.3 Scope of the Project

The current scope includes the following capabilities:

- Account registration, email/password login, logout, password hashing, and optional Google OAuth integration.
- User profile management, including name, handle, gender, biography, and avatar upload.
- Creation of expense groups with a name, type, description, and currency.
- Addition of members to a group through their EvenUp handle.
- Recording of expenses with description, amount, category, payer, and split participants.
- Automatic balance calculation and settlement suggestions.
- Recording of a settlement payment between two group members.
- Group message or chat functionality for expense-related coordination.
- Expense history with searching, category filtering, group filtering, and date filtering.
- Insights grouped by category, group, month, and recent day-level activity.

- Client-side reporting controls for PDF, PNG, and CSV-style export workflows where available in the current interface.
- Responsive styling for desktop and smaller screens.

The following items remain outside the current completed scope or require further hardening: production hosting, a production-grade secret-management process, comprehensive automated browser tests, multi-currency conversion within a single group, payment-gateway integration, push notifications, an administrator console, and a large-scale load test against a production database.

1.4 Intended Users

The primary users are small trusted groups. Typical examples are flatmates, families, travel groups, friend circles, student teams, and small event committees. Users are expected to have a modern web browser and basic familiarity with forms and web navigation. The application is not intended to replace regulated accounting software or enterprise financial systems.

1.5 Definitions, Acronyms, and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
SDLC	Software Development Life Cycle
UML	Unified Modeling Language
FR	Functional Requirement
NFR	Non-Functional Requirement
CRUD	Create, Read, Update, and Delete
API	Application Programming Interface
OAuth	Open Authorization protocol used for delegated sign-in
Group	A shared expense space containing users and expenses
Expense share	The portion of an expense assigned to a member
Balance	Net amount a member should receive or pay
Settlement	A recorded payment intended to reduce an outstanding balance
Handle	A unique public user identifier such as @alex
WBS	Work Breakdown Structure
P95 latency	The latency below which 95 percent of measured requests completed

1.6 Document Conventions

Functional requirements use identifiers such as FR-AUTH-01. Non-functional requirements use identifiers such as NFR-PERF-01. The label Implemented refers to behavior present in the current project source. The label Planned refers to a requirement that is specified for completion or hardening but is not claimed as complete.

The UML source files in srs/uml/ are the editable source of truth for the nine diagrams. The PNG files in srs/uml-rendered/ are presentation renderings generated from those Mermaid files.

2. Problem Statement

2.1 Industry Perspective

The shared-expense and bill-splitting problem appears in consumer finance, travel coordination, household management, student services, and event planning. Existing commercial applications demonstrate demand for digital expense sharing, but a student or small community project may need a transparent, self-hostable, and customisable alternative. A local institution may also prefer an application whose data model and logic can be inspected rather than relying entirely on a closed subscription service.

The industry problem is therefore not only the absence of a calculator. It is the absence of a consistent record that connects people, groups, payments, expense shares, balances, and settlement history. EvenUp treats the expense record as the central source of truth.

2.2 Social Perspective

Money disputes often arise from ambiguity rather than from the size of the amount. A person may remember paying for dinner but forget whether another member was included. A group may agree that a debt was settled but have no durable record of the payment. In a social group, these uncertainties create repeated conversations and reduce trust.

EvenUp reduces this friction by making the payer, amount, participants, category, and resulting balance visible to the group. The system supports a shared view instead of leaving one person as the sole keeper of a spreadsheet or chat message.

2.3 Institutional Perspective

Institutions such as colleges, hostels, clubs, and small organisations often manage trips, workshops, shared meals, student events, and pooled purchases. The process is frequently handled through handwritten notes, spreadsheets, or informal messaging. These approaches are difficult to audit and become less reliable as the number of transactions increases.

EvenUp can provide a lightweight record for such groups. It is not a replacement for institutional accounting, but it can serve as a controlled coordination layer for small shared-cost activities. The design also makes it possible to extend the system later with approvals, role-based access, institutional reports, and audit controls.

2.4 Limitations of the Existing Manual Approach

Manual problem	Consequence	EvenUp response
Balances are calculated by hand	Arithmetic mistakes and repeated work	Recalculate balances from stored expenses and settlements
One person maintains the record	Single point of failure and low visibility	Shared group view for members
Unequal or selected-member splits are difficult	Members may be charged incorrectly	Store explicit expense shares
Settlements are discussed in chat	No durable proof of payment	Record settlement transactions
Spending patterns are hidden	No practical insight into categories or groups	Insights and history views

Expense discussion is separate from the record	Context is lost	Group message area beside group data
Old records are difficult to search	Time is lost locating transactions	Search and filters in expense history

2.5 Proposed Solution

EvenUp proposes a browser-based system in which an authenticated user creates or joins groups, records expenses, and views live balances. Each expense is linked to a payer and one or more members. The application calculates the resulting net positions and provides settlement suggestions. Users can record completed payments, inspect expense history, and use insights to understand how group spending is distributed.

The solution is designed around three principles. First, the stored transaction record must be more authoritative than a message. Second, the calculation must be repeatable from the database state. Third, the interface must make the next action clear: add an expense, inspect a balance, or record a settlement.

3. Feasibility Study

3.1 Technical Feasibility

EvenUp is technically feasible with the selected stack. Flask provides routing, request handling, session support, and template integration. Jinja templates and CSS are sufficient for the current responsive interface. MySQL provides relational storage for users, groups, memberships, expenses, shares, messages, and settlements. The mysql-connector-python library supplies the database connection layer.

The domain is relational, which makes a normalized database appropriate. Users and groups have a many-to-many relationship through group_members. Expenses belong to groups and produce one or more expense_shares. Settlements reference a group and two users. These relationships can be represented with primary keys and foreign keys.

The system is technically feasible for an academic prototype because it can run on a modest computer with Python 3, Flask, MySQL, and a modern browser. The project has also been verified with a local HTTP smoke test and a small local performance baseline.

Technical risks include database availability, deployment configuration, secure secret management, input validation at scale, and the absence of a production reverse proxy in the current prototype. These risks are manageable through environment variables, HTTPS, database backups, automated tests, and a production WSGI deployment.

3.2 Economic Feasibility

The prototype uses open-source or freely available technologies. Flask, Jinja, MySQL Community Server, Python, and Chromium can be used without a commercial software licence for the academic project. Development can be carried out on existing laboratory or personal hardware.

The main economic costs would arise only in a production deployment. These may include a hosting server, managed database, domain name, object storage for profile images, email delivery, monitoring, and backup services. Because the first release is targeted at small groups, a low-cost deployment would be sufficient for demonstration and early evaluation.

The expected benefit is a reusable academic system that demonstrates requirements engineering, relational modelling, web development, authentication, reporting, and testing. The project is economically feasible for its intended educational and small-group use.

3.3 Operational Feasibility

The system is operationally feasible because the primary workflow is familiar: a user signs in, creates or opens a group, records an expense, and checks the resulting balance. The application uses ordinary web forms and does not require users to install a mobile application. Expense categories, groups, and settlements are visible in the same workspace.

Operational success depends on members entering complete and honest records. The system cannot independently verify that a real-world payment occurred. The settlement record is therefore a shared acknowledgement rather than a banking confirmation. The interface should continue to show the limitation clearly.

The proposed workflow requires limited training. A short demonstration can explain account creation, group creation, adding members, adding an expense, and recording a settlement. The interface can be operated by users with ordinary browser knowledge.

3.4 Feasibility Conclusion

EvenUp is technically, economically, and operationally feasible as an academic web application and as a small-group prototype. Production use would require additional security, testing, deployment, monitoring, backup, and privacy work. The current scope is appropriate for incremental completion within an academic SDLC.

4. Requirement Engineering

4.1 Requirement Elicitation Approach

Requirements were derived from the problem domain, the manual expense-sharing workflow, review of existing project pages, iterative interface design, and implementation feedback. The core user journey was decomposed into account, group, expense, balance, settlement, communication, reporting, and administration concerns.

The requirements were then checked against the current Flask routes, templates, database tables, and available evidence. This prevents the SRS from claiming that a proposed feature is already complete when it remains planned.

4.2 Functional Requirements

4.2.1 Account and Authentication Requirements

ID	Requirement	Status
FR-AUTH-01	The system shall allow a visitor to register with name, handle, email, password, and password confirmation.	Implemented
FR-AUTH-02	The system shall validate required registration fields and handle format.	Implemented
FR-AUTH-03	The system shall prevent duplicate email addresses and handles.	Implemented
FR-AUTH-04	The system shall store a password hash rather than a plain password.	Implemented
FR-AUTH-05	The system shall allow a registered user to log in and log out.	Implemented
FR-AUTH-06	The system shall protect workspace routes from unauthenticated access.	Implemented
FR-AUTH-07	The system may support Google OAuth when credentials are configured.	Implemented conditionally
FR-AUTH-08	The system shall display a useful validation or authentication error.	Implemented

4.2.2 Profile Requirements

ID	Requirement	Status
FR-PROF-01	The user shall view and edit profile details.	Implemented
FR-PROF-02	The user shall maintain a unique public handle.	Implemented
FR-PROF-03	The user shall optionally upload a supported profile image.	Implemented

FR-PROF-04	The system shall reject unsupported image types and files above the configured size limit.	Implemented
FR-PROF-05	Profile information shall be available to group members where required by the interface.	Implemented

4.2.3 Group Requirements

ID	Requirement	Status
FR-GRP-01	An authenticated user shall create a group with name, type, currency, and description.	Implemented
FR-GRP-02	The system shall list groups visible to the current user.	Implemented
FR-GRP-03	A group owner shall add another registered user by handle.	Implemented
FR-GRP-04	A user shall view group members and group details.	Implemented
FR-GRP-05	Group records shall be isolated so that a user cannot access an unrelated group through a guessed identifier.	Implemented through membership checks

4.2.4 Expense Requirements

ID	Requirement	Status
FR-EXP-01	A group member shall record an expense description and amount.	Implemented
FR-EXP-02	A member shall select a payer from the group members.	Implemented
FR-EXP-03	A member shall select a category.	Implemented
FR-EXP-04	A member shall split an expense equally among all members or selected members.	Implemented
FR-EXP-05	The system shall validate that the amount is positive and the selected split is valid.	Implemented
FR-EXP-06	The system shall store expense shares separately from the expense header.	Implemented
FR-EXP-07	The system shall show the resulting expense in group history.	Implemented
FR-EXP-08	The system shall show all accessible expense history in the Expenses workspace.	Implemented

4.2.5 Balance and Settlement Requirements

ID	Requirement	Status
FR-BAL-01	The system shall calculate how much each member paid.	Implemented
FR-BAL-02	The system shall calculate how much each member should bear from expense shares.	Implemented
FR-BAL-03	The system shall include recorded settlements in the net balance.	Implemented
FR-BAL-04	The system shall generate settlement suggestions from positive and negative balances.	Implemented
FR-BAL-05	A user shall record a payment from one member to another.	Implemented
FR-BAL-06	The group view shall show current balances and settlement history.	Implemented

4.2.6 Collaboration Requirements

ID	Requirement	Status
FR-COL-01	Group members shall post messages in the group message area.	Implemented
FR-COL-02	Messages shall display the author and creation time.	Implemented
FR-COL-03	A user shall not post to a group to which the user does not belong.	Implemented through group access checks

4.2.7 Insights and Reporting Requirements

ID	Requirement	Status
FR-REP-01	The system shall aggregate spending by category.	Implemented
FR-REP-02	The system shall aggregate spending by group.	Implemented
FR-REP-03	The system shall provide a recent monthly trend dataset.	Implemented
FR-REP-04	The system shall provide recent daily spending data for the insights view.	Implemented
FR-REP-05	The Expenses view shall support text search.	Implemented
FR-REP-06	The Expenses view shall support category, group, and date filters.	Implemented
FR-REP-07	The system shall provide a filtered total for the current expense history view.	Implemented
FR-REP-08	The user interface shall provide report export controls for PDF,	Implemented in interface; production export verification remains planned

	PNG, and CSV-style outputs where enabled by the current page.	

4.2.8 Navigation and Usability Requirements

ID	Requirement	Status
FR-UX-01	The landing page shall explain the product and provide calls to action.	Implemented
FR-UX-02	Authenticated users shall see Home, Workspace, and Sign out actions.	Implemented
FR-UX-03	The interface shall use responsive layouts for smaller screens.	Implemented in CSS
FR-UX-04	Error and success messages shall be visible after relevant actions.	Implemented

4.3 Non-Functional Requirements

ID	Requirement	Verification approach
NFR-SEC-01	Passwords shall not be stored as plain text.	Code inspection and registration test
NFR-SEC-02	Protected routes shall require a valid session.	Route test without session
NFR-SEC-03	User input shall be validated before database writes.	Validation test and code review
NFR-SEC-04	Database queries shall use parameterized values.	Code inspection
NFR-PERF-01	Public HTML endpoints should respond quickly in the local baseline.	Timed HTTP test
NFR-PERF-02	The application shall respond successfully under a small concurrent request sample.	25-request concurrency sample
NFR-REL-01	A successful expense write shall commit the expense and its shares consistently.	Database transaction test
NFR-REL-02	A failed request shall return a clear error or redirect rather than silently succeeding.	Negative tests
NFR-USAB-01	Core workflows shall be understandable without specialist training.	Task walkthrough
NFR-USAB-02	The interface shall provide visible feedback for success and failure.	Browser inspection
NFR-MAINT-01	Requirements, route structure, and database entities shall be documented.	SRS and source review
NFR-MAINT-02	UML diagrams shall be editable rather than supplied only as screenshots.	Review .mmd files

NFR-PORT-01	The application shall run on a standard Python and MySQL environment.	Installation and smoke test

4.4 Hardware and Software Requirements

Development environment

The development environment requires a computer capable of running Python 3, Flask, MySQL, a modern browser, and a code editor. A practical baseline is a dual-core processor, 4 GB RAM, at least 1 GB of free storage for the application and dependencies, and a local or reachable MySQL server.

End-user environment

The end user requires a modern browser such as Chromium, Chrome, Firefox, Edge, or Safari. The browser must support HTML5, CSS, JavaScript used by the interface, and secure cookies. The user does not need a separate installation if the application is deployed to a reachable server.

4.5 Data Requirements

The core database consists of users, groups_tbl, group_members, expenses, expense_shares, group_messages, settlements, and contact_messages. Foreign keys preserve the relationships between users, groups, transactions, and messages. Group and expense data should be backed up before production use. Uploaded images require a retention and privacy policy in a production environment.

4.6 Requirement Traceability Summary

The problem statement is addressed by the expense, share, balance, and settlement requirements. The social concerns are addressed by group visibility, chat, history, and settlement records. The institutional concerns are addressed by persistent records, categories, reports, and future-ready relational storage. The requirements are represented in the UML use-case, class, sequence, activity, state, component, deployment, ER, and object diagrams in Section 6.

5. Software Development Life Cycle Planning

5.1 Selected SDLC Model

The project follows an iterative and incremental SDLC model. The model is suitable because the interface, expense calculations, reporting needs, and project scope are refined through feedback. Each increment can produce a usable part of the system while later increments improve validation, visual design, export support, and deployment readiness.

The iterations are organised as follows: problem analysis and scope, requirements, database and architecture, authentication, groups and members, expenses and balances, insights and reporting, testing and evidence, and final documentation. The current project has completed a working prototype increment and continues to require hardening and deployment evaluation.

5.2 Work Breakdown Structure

WBS	Work package	Main outputs
1.0	Project initiation	Problem statement, objectives, scope, stakeholders
2.0	Requirement engineering	Functional requirements, NFRs, traceability
3.0	System design	Architecture, database schema, UML models, UI plan
4.0	Account module	Registration, login, logout, optional OAuth, profile
5.0	Group module	Group creation, membership, group detail page
6.0	Expense module	Expense entry, split shares, history, filters
7.0	Balance module	Balance calculation, suggestions, settlement records
8.0	Collaboration module	Group messages and history
9.0	Insights module	Category, group, monthly, and daily aggregations
10.0	Reporting and exports	PDF/PNG/CSV interface and report evidence
11.0	Testing	Smoke tests, validation tests, performance baseline, browser evidence
12.0	Deployment preparation	Environment configuration, production checklist, backup plan
13.0	Evaluation and documentation	SRS, screenshots, traceability, future scope

5.3 Project Timeline — Gantt Chart

Workstream	Planned period	Status
Problem study and scope	1–7 July 2026	Complete
Requirement engineering	8–17 July 2026	Complete
Database, architecture, UML, and interface design	15 July–5 August 2026	Complete
Authentication and profiles	1–12 August 2026	Complete
Groups and membership	13–22 August 2026	Complete
Expenses and balances	23 August–5 September 2026	Complete
Insights and expense history	6–17 September 2026	Complete
Reporting and visual refinement	18–27 September 2026	Active
Functional, negative, and performance testing	1–15 September 2026	Active / baseline captured
Deployment preparation	18–24 September 2026	Planned
Final SRS and evaluation	25 September–4 October 2026	Active

5.4 Milestones

Milestone	Acceptance condition
M1 — Requirements baseline	Scope, stakeholders, FRs, and NFRs reviewed
M2 — Design baseline	Database, architecture, and UML source approved
M3 — Working prototype	User can authenticate, create a group, and record an expense
M4 — Reporting increment	User can inspect history, filters, and insights
M5 — Verification baseline	Smoke tests and local performance evidence captured
M6 — Academic submission	SRS, editable diagrams, screenshots, and limitations included

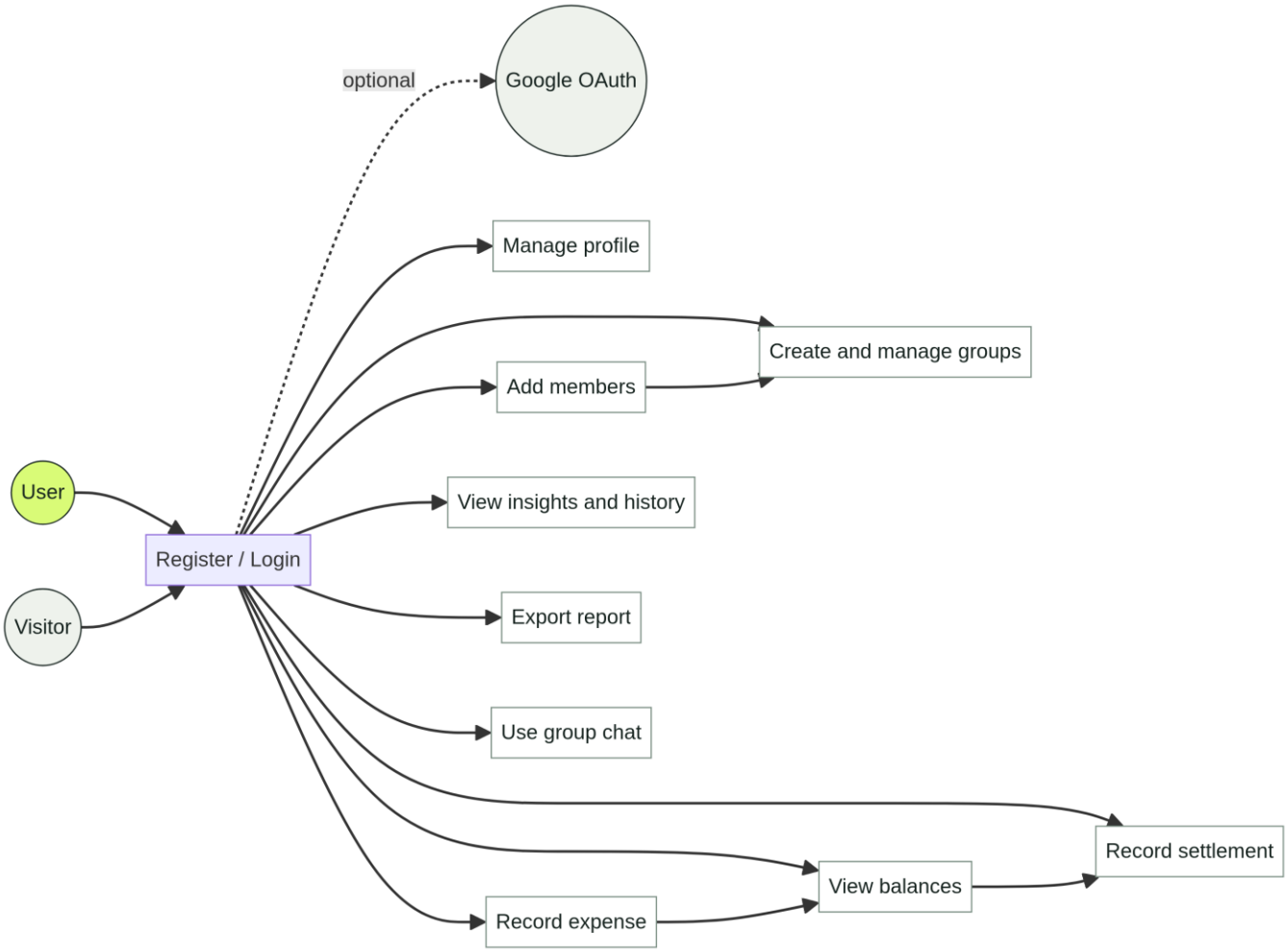
6. System Modelling Using UML

The nine diagrams below are rendered for readability, while each diagram remains editable in Mermaid source form. The source file links are included with every figure.

6.1 Use-Case Diagram

The use-case model identifies the visitor, authenticated user, and optional Google OAuth provider. The authenticated user performs account, group, expense, balance, settlement, collaboration, history, insight, and report actions.

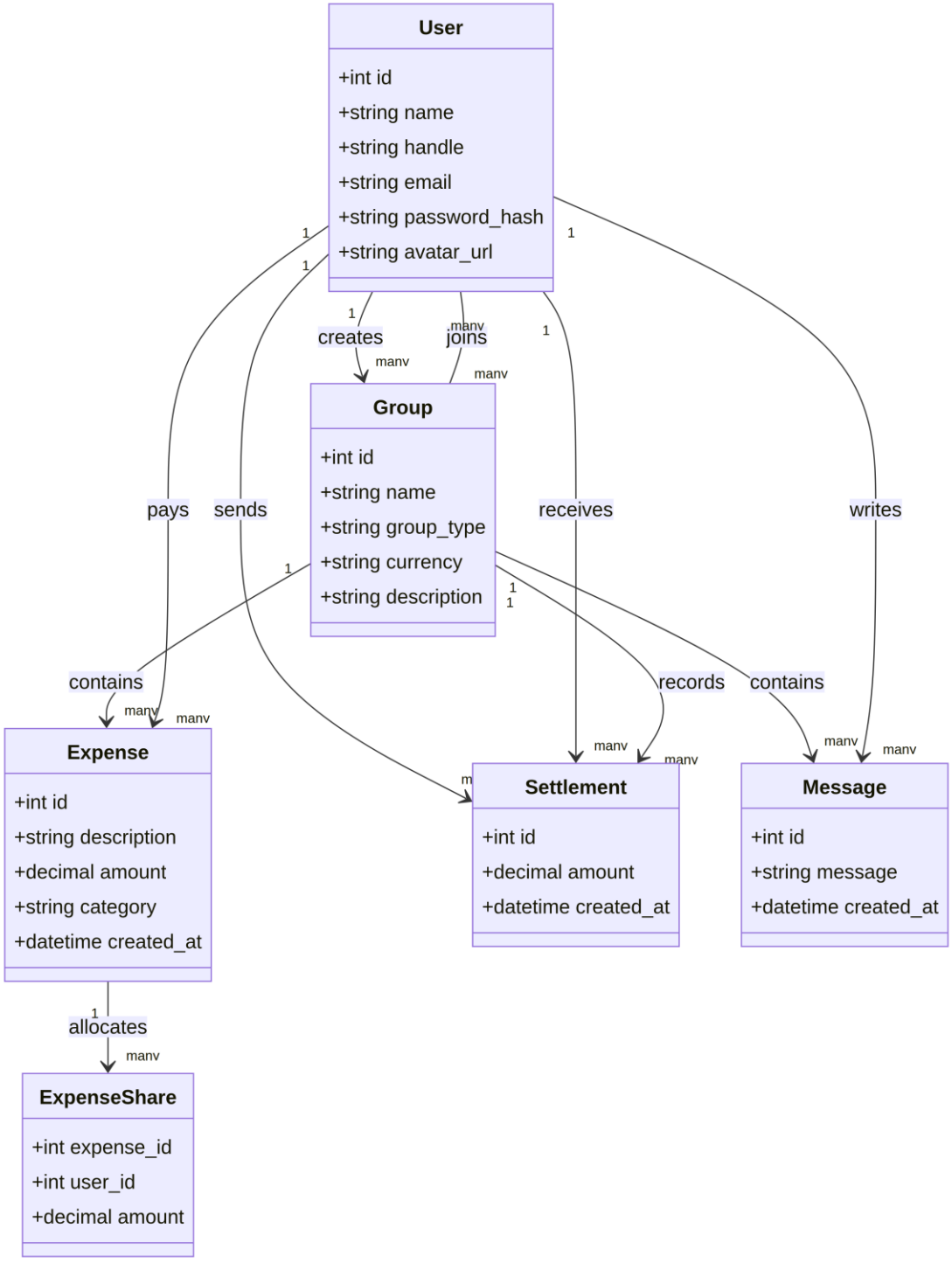
EvenUp use-case diagram



6.2 Class Diagram

The class model represents users, groups, expenses, expense shares, settlements, and messages. It shows the many-to-many membership relation and the one-to-many transaction relations.

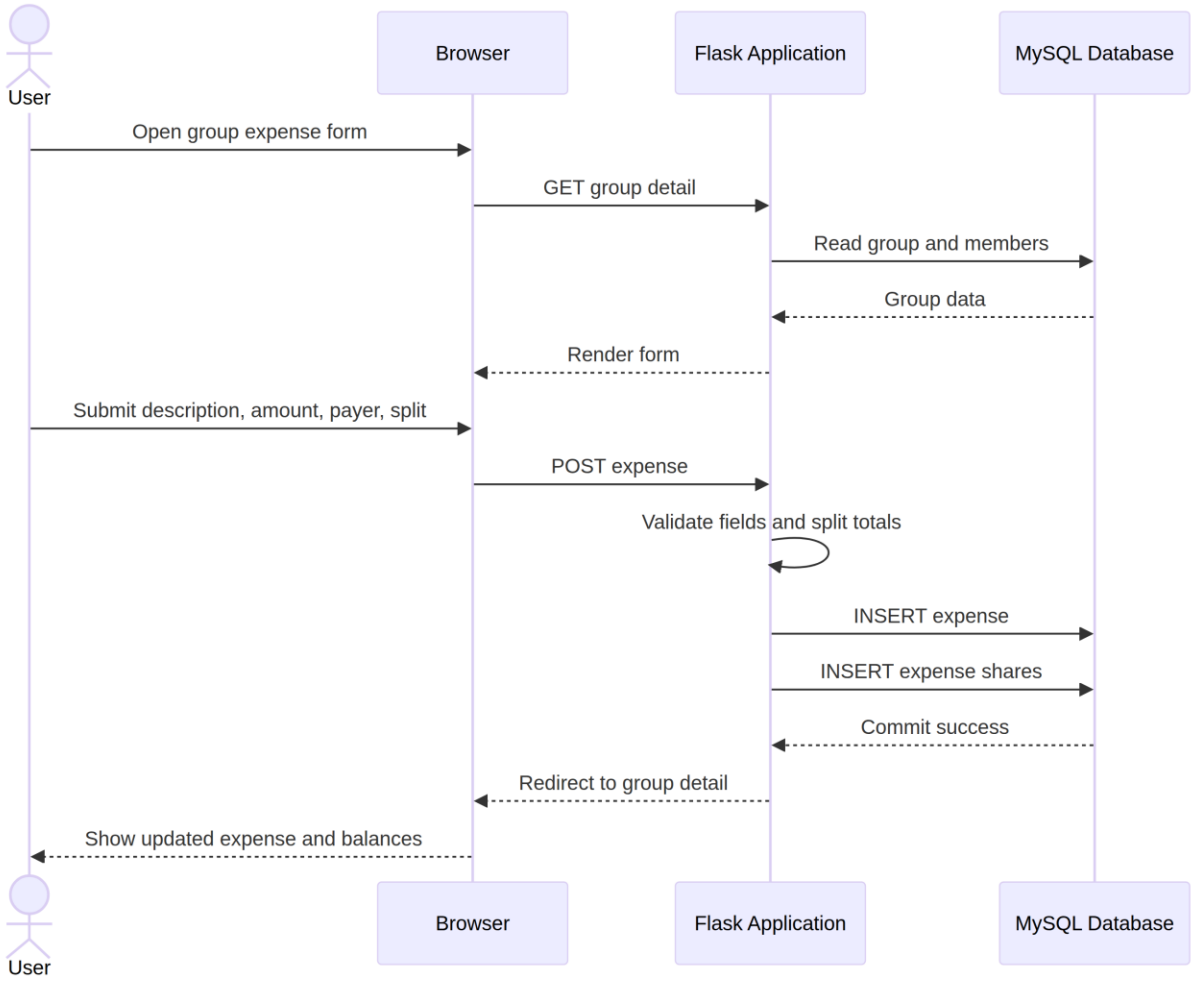
EvenUp use-case diagram



6.3 Sequence Diagram -Add Expense

The sequence begins when a user opens a group, continues through form submission and validation, and ends when the browser displays the updated expense and balances.

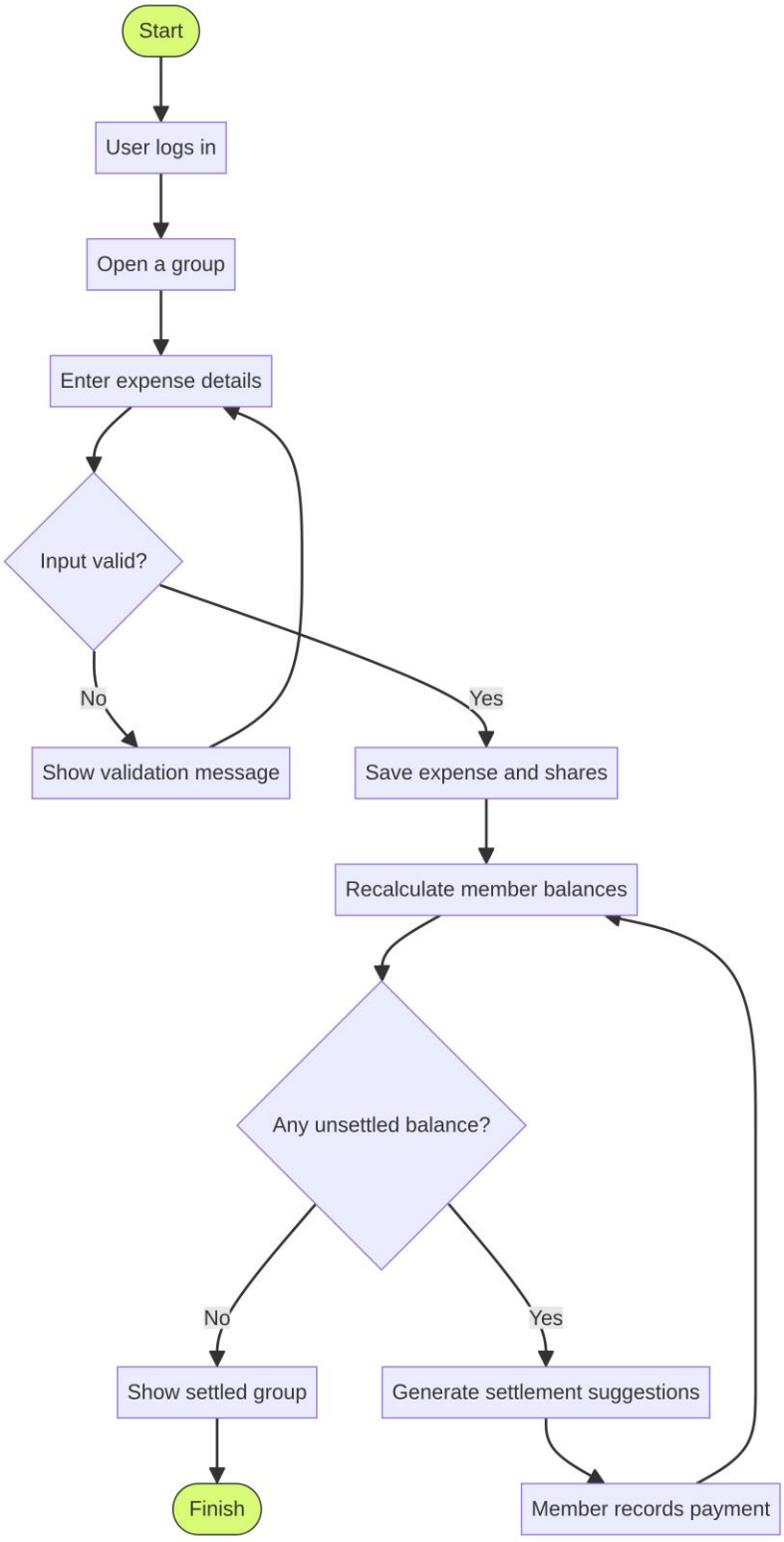
Add-expense sequence diagram



6.4 Activity Diagram — Settlement Workflow

The activity model shows validation, persistence, balance recalculation, settlement suggestion generation, payment recording, and termination when the group is settled.

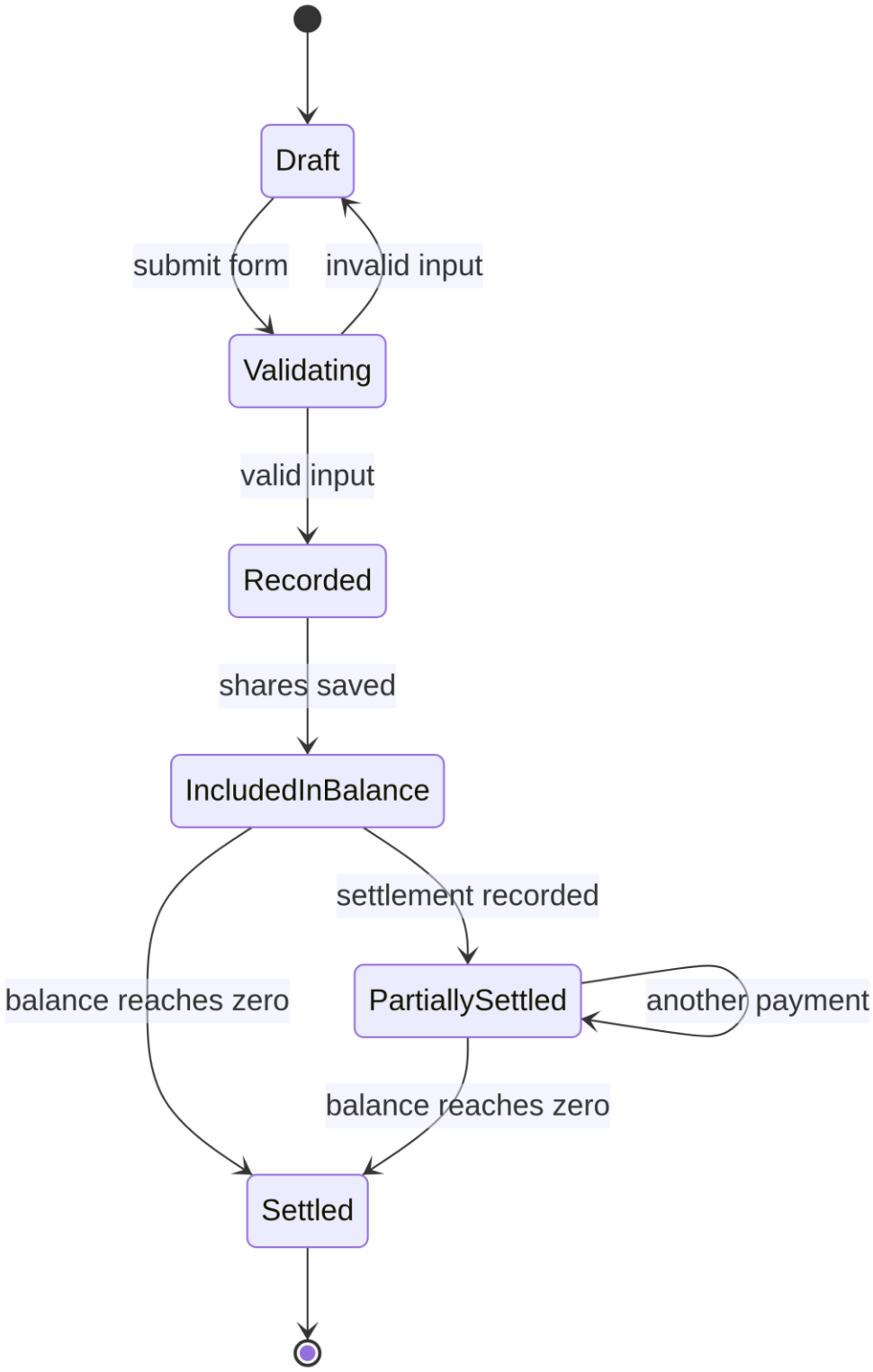
Balance Settlement Diagram



6.5 State Diagram — Expense Lifecycle

An expense moves from draft to validation, recording, balance inclusion, partial settlement, and final settlement. Invalid input returns the workflow to the draft state.

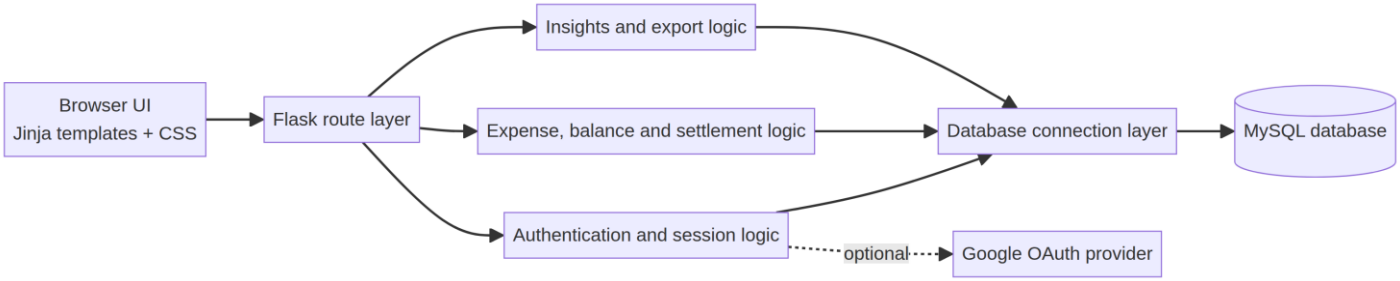
Expense state diagram



6.6 Component Diagram

The component model separates the browser UI, Flask routes, authentication, domain logic, reports, database layer, MySQL database, and optional OAuth provider.

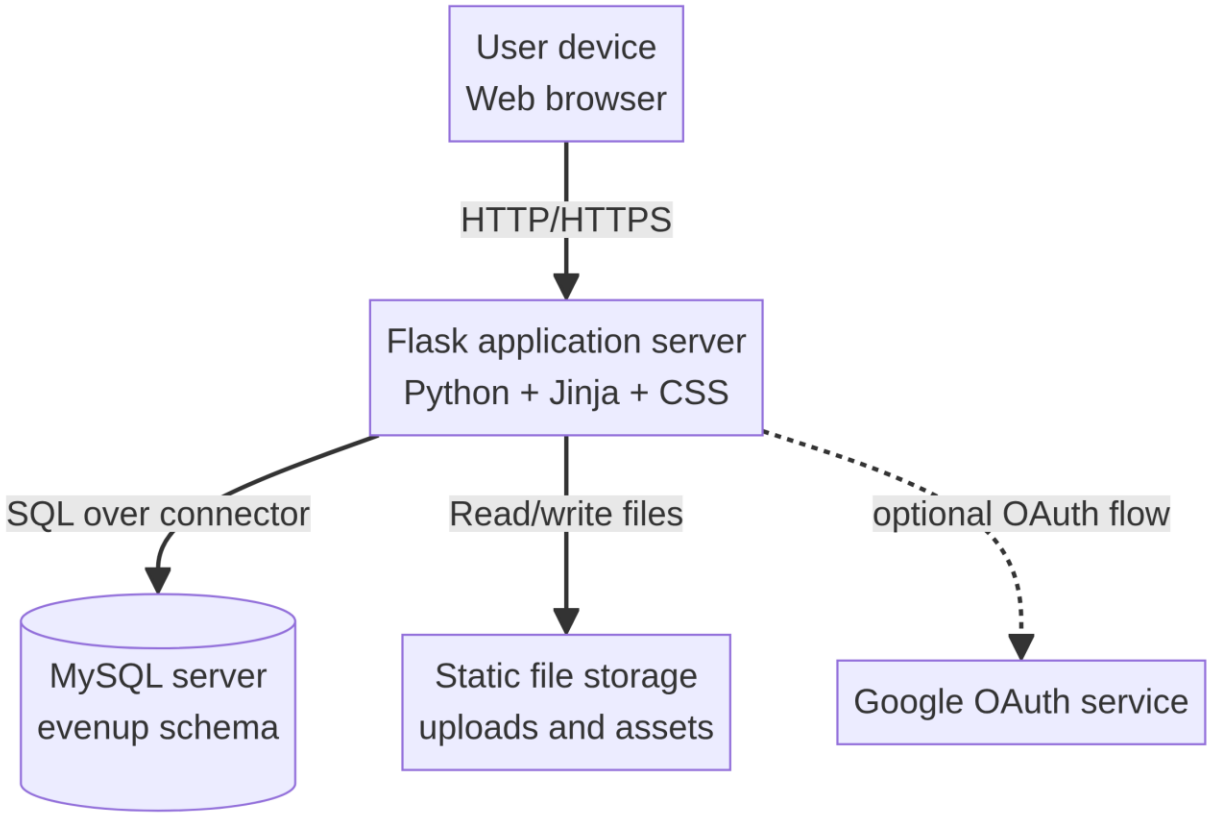
Component diagram



6.7 Deployment Diagram

The deployment model shows a user device communicating with the Flask application server. The application connects to MySQL, static file storage, and optionally Google OAuth.

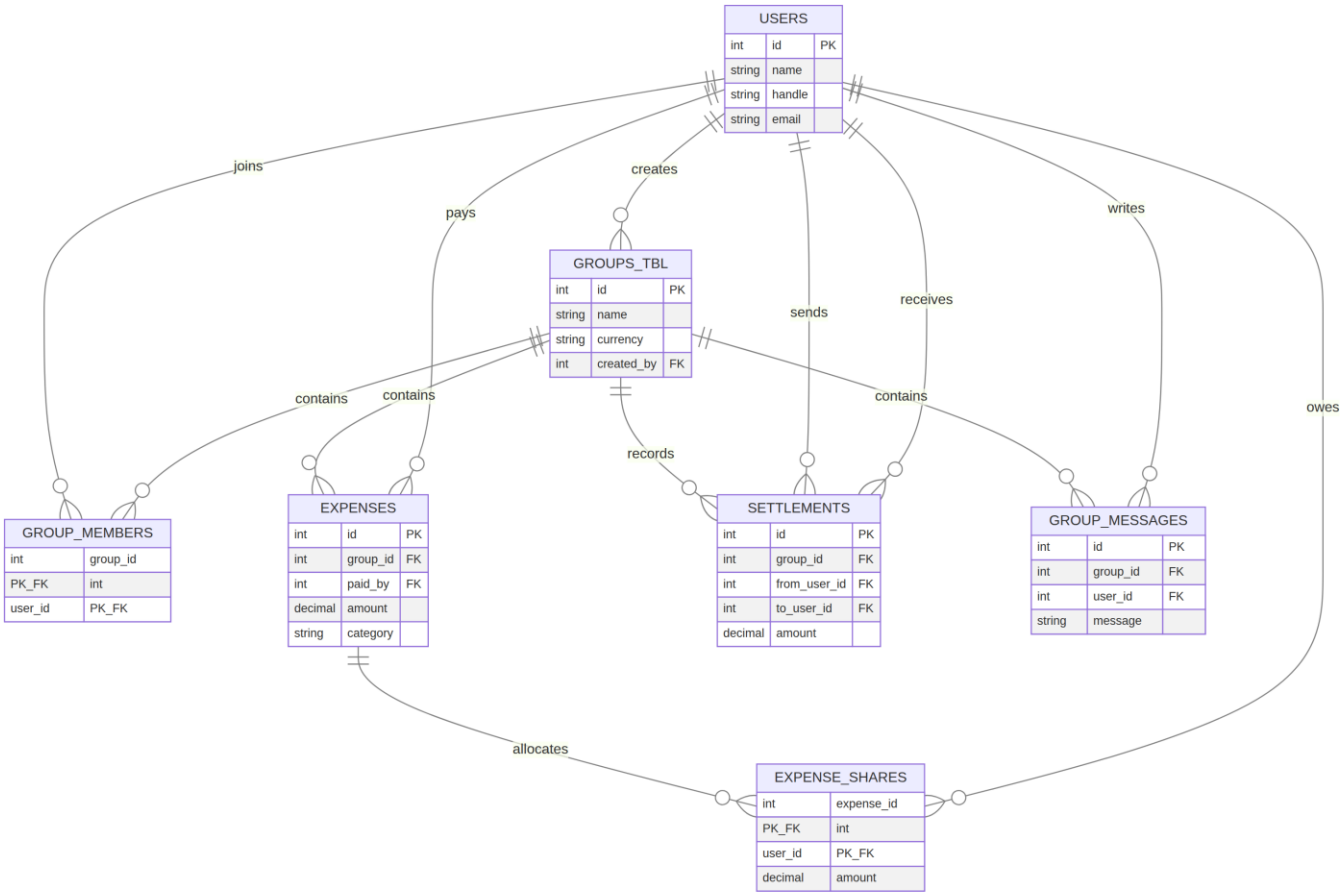
Deployment diagram



6.8 Entity–Relationship Diagram

The ER model follows the implemented database tables and their foreign-key relationships. It provides the database view required for implementation and evaluation.

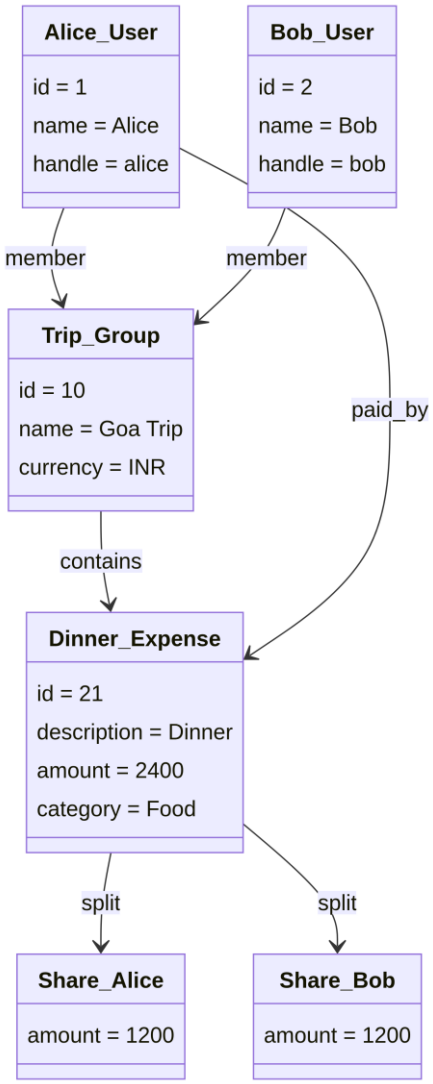
Entity–relationship diagram



6.9 Object Diagram — Sample Runtime Snapshot

The object snapshot shows a small Goa Trip group in which Alice and Bob share a dinner expense. Alice is the payer and the expense is split into two stored shares.

Object snapshot diagram



7. System Architecture Design

7.1 Architecture Overview

EvenUp uses a layered web architecture. The browser renders responsive pages and submits forms. Flask receives HTTP requests and performs routing, session checks, validation, domain calculations, database reads, and database writes. Jinja renders server-side HTML templates. MySQL stores persistent application data. Static files contain CSS, uploaded profile images, generated logo assets, and client-side resources.

The application is currently a modular Flask monolith. This is an appropriate architecture for the project because the user workflows are related, the deployment footprint is small, and the database transaction boundaries are easier to understand in one application. The architecture can later be split into separate services only if scale or team structure requires it.

7.2 Current Project Structure

```
EvenUp/
├── app.py
├── db.py
├── requirements.txt
├── templates/
│   ├── base.html
│   ├── landing.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   ├── workspace.html
│   ├── create_group.html
│   ├── group_detail.html
│   ├── expenses.html
│   ├── insights.html
│   └── profile.html
├── static/
│   ├── style.css
│   ├── assets/
```

7.3 Frontend Design

The frontend consists of Jinja HTML templates and a shared stylesheet. The base template supplies the title, favicon, global navigation, flash messages, and shared assets. The landing template provides the public product explanation and calls to action. Authenticated workspace templates provide dashboard, group, expense, insight, profile, and group-detail views.

The UI uses a dark forest-green and light neutral palette with lime accent actions. Reusable patterns include navigation links, cards, forms, filter controls, history rows, status messages, summary metrics, and responsive grid layouts. The home and workspace navigation are intentionally related so that users can return from the workspace to the landing page.

7.4 Backend Design

The backend is implemented in app.py. Authentication is provided by registration, password hashing, login, logout, a login-required decorator, and optional Google OAuth. The route layer calls database helpers and renders templates. Expense and settlement calculations are performed from stored transaction data.

The database layer is implemented in db.py. It configures MySQL from environment values and creates the required tables when a connection is initialized. The schema uses foreign keys to preserve relationships and cascading deletes where appropriate.

7.5 API and Route Structure

EvenUp currently uses server-rendered routes rather than a separate JSON REST service. The main route structure is:

Route	Method	Purpose
/	GET	Public landing page
/register	GET, POST	Registration
/login	GET, POST	Login
/logout	GET	End session
/login/google	GET	Start optional Google OAuth
/auth/google/callback	GET	OAuth callback
/dashboard	GET	Authenticated summary
/groups	GET	Group workspace
/create-group	GET, POST	Create group
/groups/<id>	GET	Group detail
/groups/<id>/add-member	POST	Add a member
/groups/<id>/add-expense	POST	Add an expense and shares
/groups/<id>/messages	POST	Add group message
/groups/<id>/settle	POST	Record settlement
/expenses	GET	Searchable expense history
/insights	GET	Aggregated insights
/profile	GET, POST	View and update profile
/set-handle	GET, POST	Complete handle setup
/api/user/<handle>	GET	Find a user by handle
/contact	POST	Store a public contact message

The /expenses route accepts query parameters for search, category, group, start date, and end date. The /insights route prepares category totals, group totals, recent monthly totals, and recent daily totals.

7.6 Database Design

The users table stores identity, credentials, profile data, and avatar information. groups_tbl stores group metadata. group_members resolves the many-to-many relation between users and groups. expenses stores the expense header and payer. expense_shares stores each member's allocated share. settlements records payments between members. group_messages supports group communication. contact_messages stores public contact-form submissions.

The database design separates the expense header from the expense allocation. This is important because the payer and the people bearing the cost are not always the same. The design also makes it possible to reconstruct balances from expense and settlement records.

7.7 Security and Privacy Architecture

The system uses password hashing through Werkzeug rather than storing plain passwords. Protected routes check the session before returning workspace data. Database statements use parameterized values. Upload validation restricts file extensions and configured file size. A production deployment must add HTTPS, secure cookie settings, CSRF protection, rate limiting, stricter MIME validation, secret rotation, database backups, and a documented privacy policy.

8. Implementation, Testing, Deployment, and Evaluation

8.1 Implementation Status

The project is implemented as a Flask application with Jinja templates, CSS, MySQL database access, and static assets. The current codebase contains public landing, login, registration, profile, dashboard, groups, expense history, insights, group detail, and create-group templates.

The implementation includes account management, group creation, membership, expense recording, balance computation, settlement recording, messages, expense filtering, insights aggregation, logo assets, responsive styling, and the updated authenticated navigation.

Export controls and interface improvements are present in the reporting views. Final production-grade export validation and browser coverage remain recommended next steps. The project should therefore be described as a functioning academic prototype rather than a finished commercial product.

8.2 Testing Strategy

Testing is divided into unit-level reasoning, route smoke tests, validation tests, database checks, browser walkthroughs, and performance tests. The aim is to verify both the correctness of calculations and the usability of the complete user flow.

8.2.1 Functional test cases

ID	Test case	Expected result	Evidence/status
TC-01	Open /	Landing page returns HTTP 200	Passed
TC-02	Open /login	Login page returns HTTP 200	Passed
TC-03	Open /register	Registration page returns HTTP 200	Passed
TC-04	Submit incomplete registration	Validation message is shown	Implemented; browser regression recommended
TC-05	Submit mismatched passwords	Registration is rejected	Implemented
TC-06	Access /dashboard without session	User is redirected to login	Implemented through decorator
TC-07	Create a group with valid fields	Group is stored and visible	Implemented; requires configured MySQL
TC-08	Add an expense	Expense and shares are stored	Implemented; requires configured MySQL
TC-09	Use equal split	Each selected member receives an equal share	Implemented
TC-10	Use selected-member split	Only selected members receive shares	Implemented
TC-11	Record settlement	Settlement is stored and balance changes	Implemented
TC-12	Filter expense history	Matching history and filtered total are shown	Implemented
TC-13	Open insights	Category, group, monthly, and daily data are rendered	Implemented

TC-14	Submit unsupported profile image	Upload is rejected	Implemented

8.2.2 Code and template validation

The current implementation was validated by compiling app.py and db.py with Python syntax checks and loading the key Jinja templates. The templates validated for the current submission are base.html, landing.html, login.html, and register.html. Mermaid sources were rendered successfully into nine PNG diagram figures.

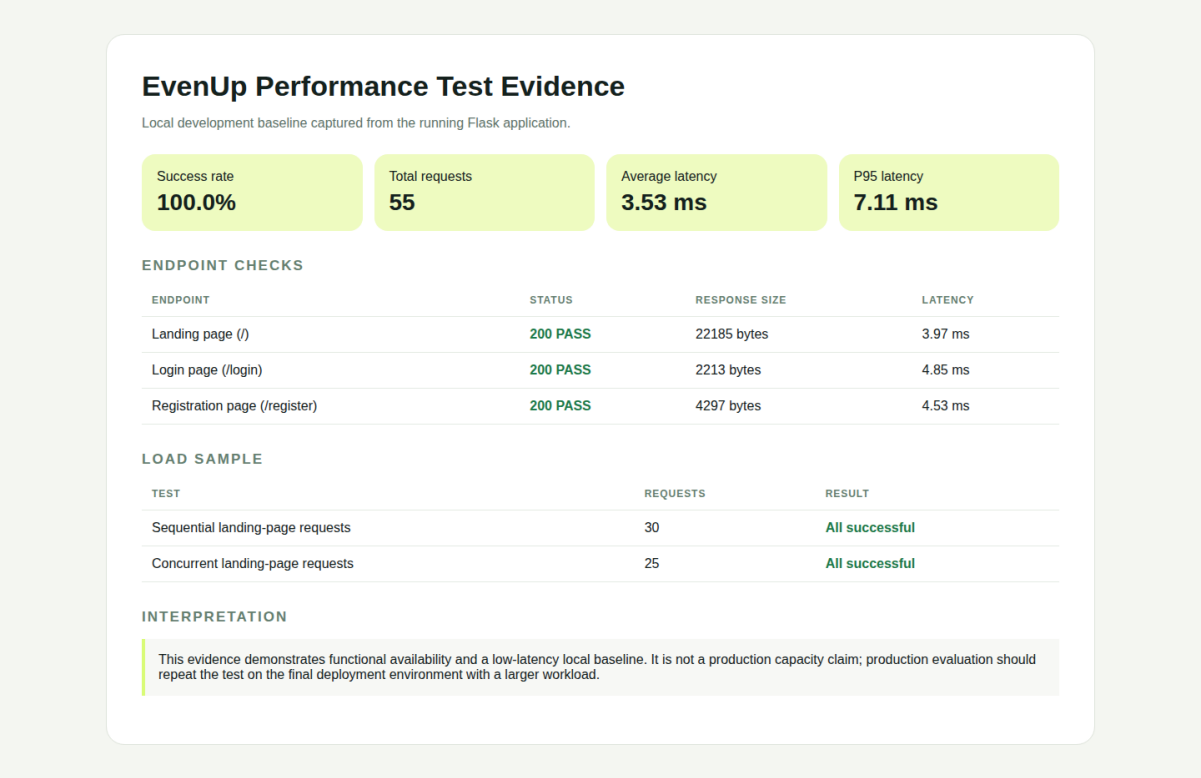
8.3 Performance Testing and Evidence

A reproducible performance script is included at tools_srs_evidence.py. It checks the landing, login, and registration endpoints, then performs 30 sequential landing-page requests and 25 concurrent landing-page requests using five workers. The test was run against the local Flask development server at http://127.0.0.1:5000.

The measured result was 55 successful responses out of 55 requests, for a 100.00 percent success rate. The average latency was 3.53 ms, the median latency was 1.84 ms, the P95 latency was 7.11 ms, and the maximum observed latency was 7.93 ms. Individual endpoint checks returned HTTP 200 for the landing page, login page, and registration page.

These figures are useful evidence that the prototype responds correctly and quickly in the local environment. They are not a production benchmark. A production evaluation should repeat the test with the actual WSGI server, deployment network, production database, realistic payloads, authentication flows, and a larger workload.

Performance testing evidence



EvenUp Performance Test Evidence

Local development baseline captured from the running Flask application.

Success rate	Total requests	Average latency	P95 latency
100.0%	55	3.53 ms	7.11 ms

ENDPOINT CHECKS

ENDPOINT	STATUS	RESPONSE SIZE	LATENCY
Landing page (/)	200 PASS	22185 bytes	3.97 ms
Login page (/login)	200 PASS	2213 bytes	4.85 ms
Registration page (/register)	200 PASS	4297 bytes	4.53 ms

LOAD SAMPLE

TEST	REQUESTS	RESULT
Sequential landing-page requests	30	All successful
Concurrent landing-page requests	25	All successful

INTERPRETATION

This evidence demonstrates functional availability and a low-latency local baseline. It is not a production capacity claim; production evaluation should repeat the test on the final deployment environment with a larger workload.

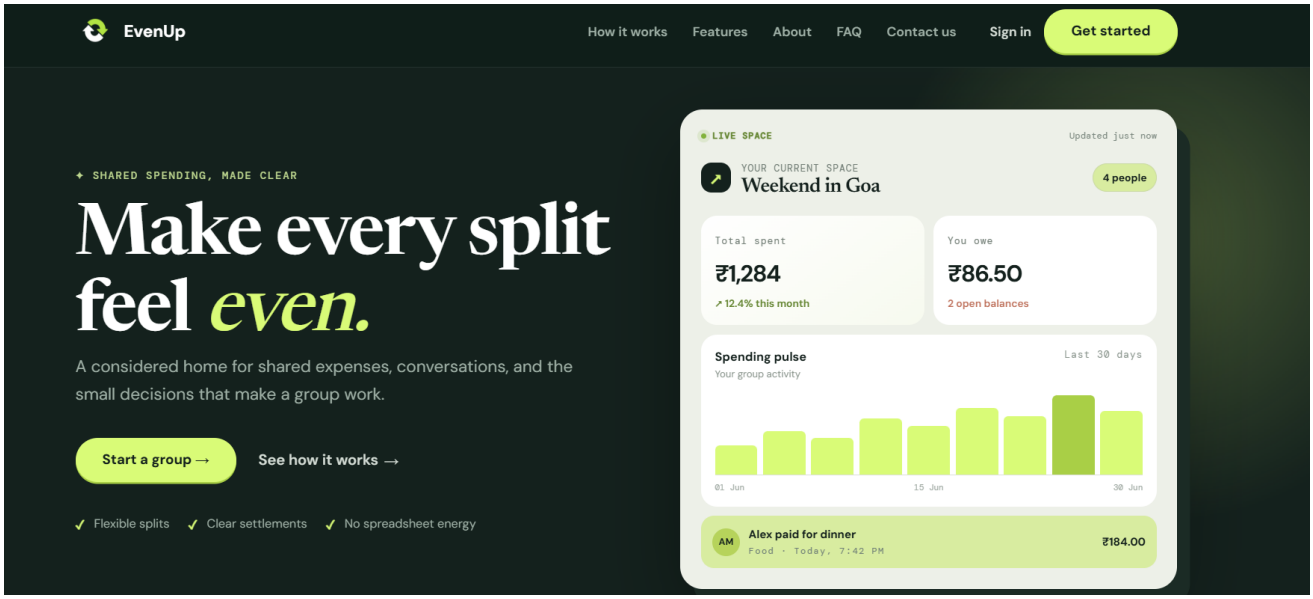
The raw result is available as performance_results.json, with a short interpretation in performance_summary.md. The rendered evidence page is available as performance_evidence.html.

8.4 Working-System Screenshots

The following screenshots document working parts of the current system. The landing page, login page, and registration page were captured from the running local Flask application. The workspace analytics screenshot was supplied during the project review and records the implemented insights layout.

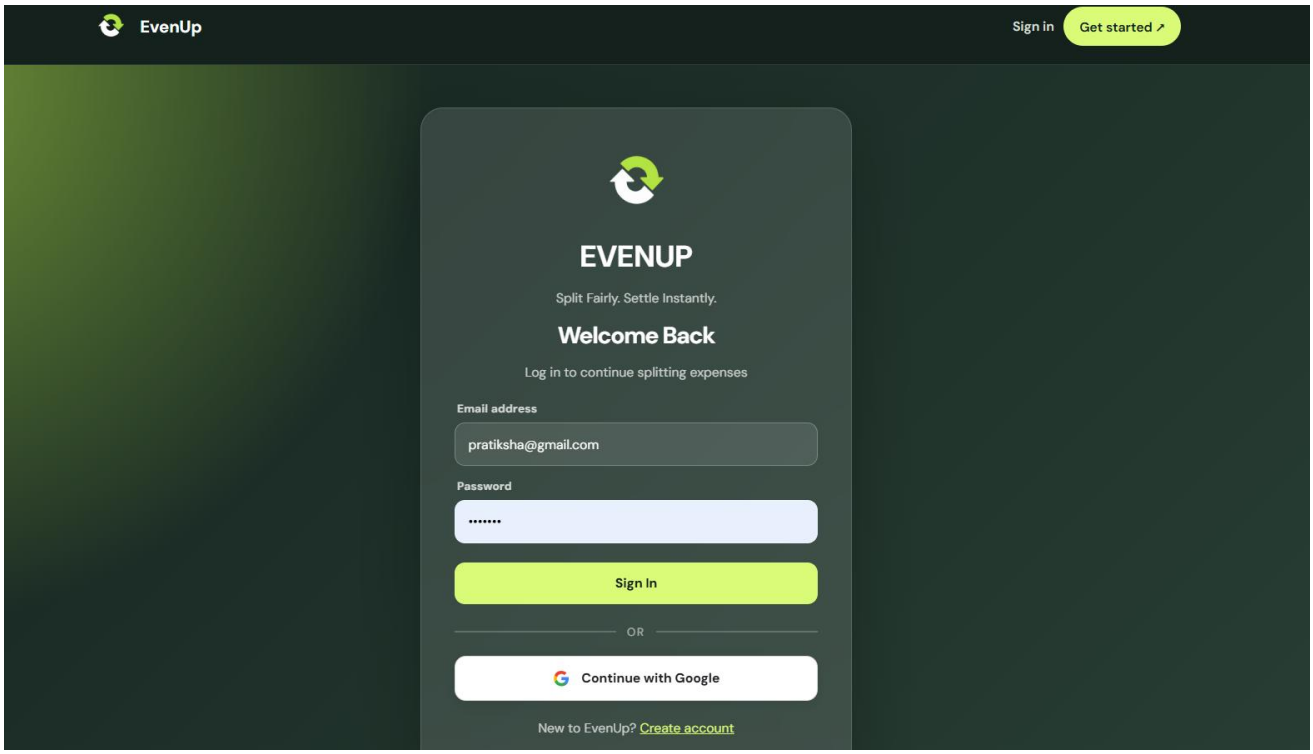
Landing page

EvenUp landing page working screenshot



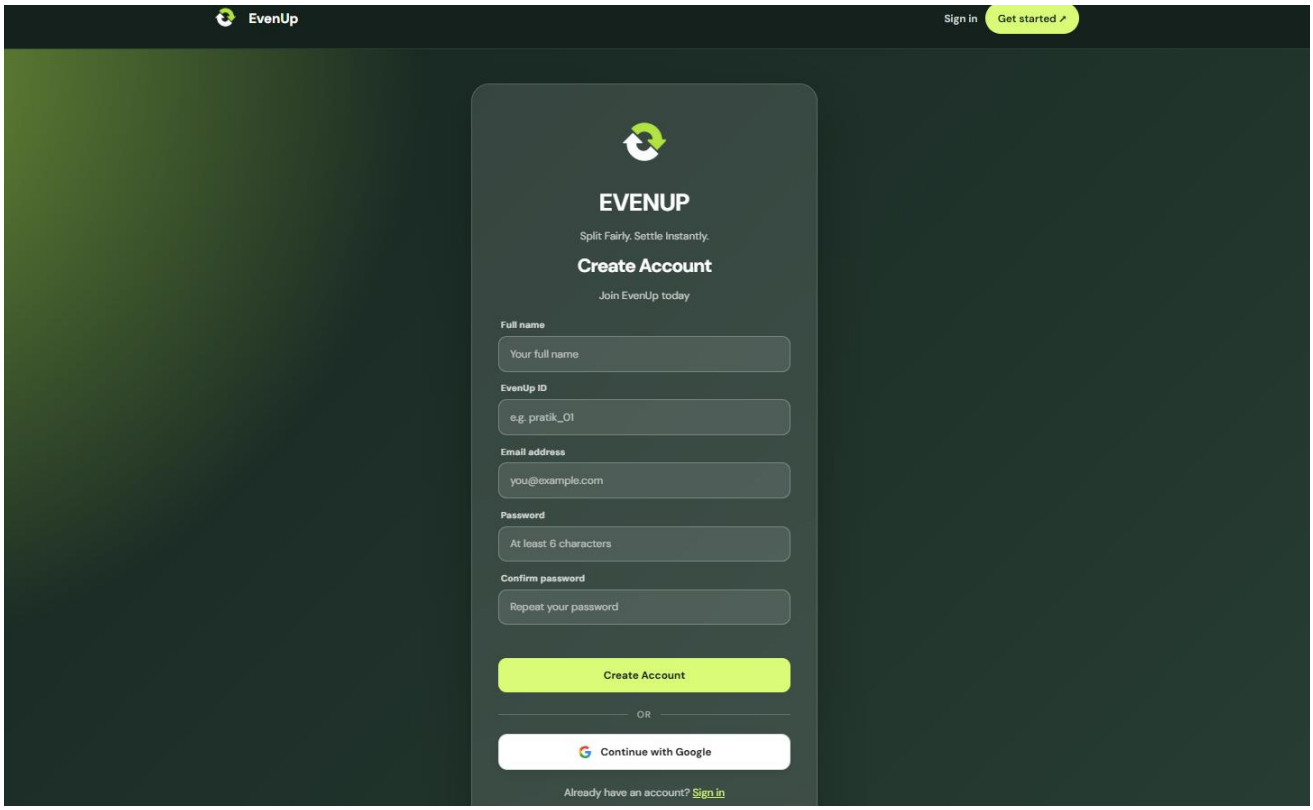
Login page

EvenUp login page working screenshot



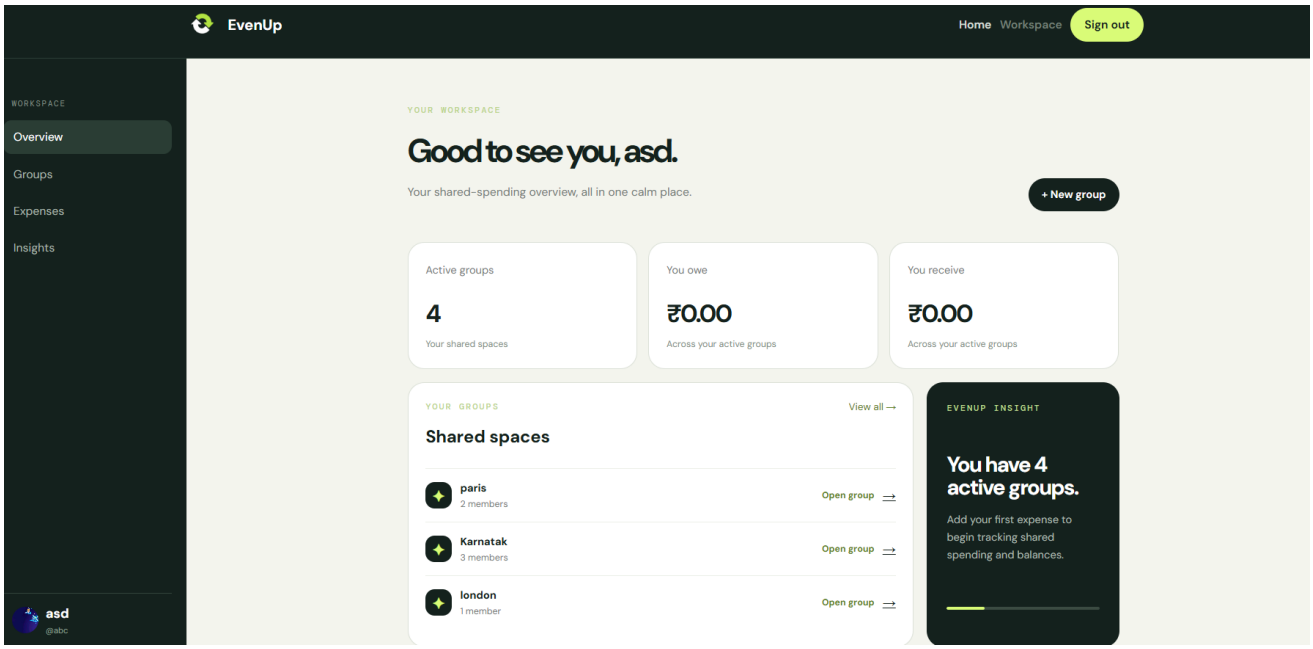
Registration page

EvenUp registration page working screenshot



Workspace page

EvenUp workspace screenshot



The screenshots are evidence of the interface state at the time of documentation. They do not replace functional tests or prove that every database workflow is production-ready.

8.5 Deployment Plan

Development deployment

1. Install Python 3 and MySQL.
1. Create a Python virtual environment.
1. Install the packages listed in requirements.txt.
1. Create the evenup MySQL database and user.
1. Set MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DATABASE, and SECRET_KEY as environment variables.
1. Run the Flask application with a development command for local testing.
1. Open the application in a supported browser.

Production deployment

A production deployment should use a WSGI server such as Gunicorn behind Nginx or an equivalent managed service. HTTPS should be enabled. Production secrets must be stored outside the source tree. The MySQL database must be backed up. Static and uploaded files should use a controlled storage strategy. Logs, error monitoring, health checks, rate limiting, and database migration management should be added before public release.

8.6 Evaluation Against Objectives

Objective	Evaluation
Reduce manual arithmetic	Achieved in the implemented expense and balance logic
Give members a shared expense record	Achieved through groups, expenses, shares, history, and messages
Support practical unequal participation	Achieved through selected-member sharing
Make settlements explicit	Achieved through balance suggestions and settlement records
Provide spending insight	Achieved through category, group, monthly, and daily aggregation
Provide a usable web interface	Achieved for the current prototype and captured in screenshots
Demonstrate maintainable engineering	Partially achieved; SRS and editable UML are provided, while automated regression coverage should be expanded
Demonstrate production readiness	Not yet achieved; deployment hardening and larger tests remain future work

8.7 Known Limitations and Risks

The current system assumes a trusted small group and does not independently verify a real-world payment. Currency conversion is not implemented within a single group. The system has no administrator role, no full audit-log interface, no payment gateway, and no push notification service. Production deployment must strengthen CSRF protection, secure cookies, secret management, upload validation, rate limiting, database migrations, and backups.

The local performance evidence does not include the MySQL-heavy authenticated workflows or a production network. It should be reported as a baseline rather than as a capacity guarantee. The next evaluation should measure login, group detail, expense insertion, insights aggregation, and concurrent database access.

9. Conclusion and Future Scope

9.1 Conclusion

EvenUp is a feasible web-based solution to the recurring problem of shared-expense calculation and settlement. The project connects the full core workflow: account access, group formation, expense entry, share allocation, balance calculation, settlement recording, history, communication, and insight generation. Its relational data model reflects the real structure of the problem and its layered Flask architecture is suitable for the current academic scale.

The project is sufficiently implemented to demonstrate a working prototype. The SRS records the design and the current evidence honestly. It also distinguishes the work that remains before a production release, including stronger automated tests, deployment hardening, privacy controls, database operations, and larger-scale performance validation.

9.2 Future Scope

Future releases can add multi-currency support, recurring expenses, receipt image attachment, reminders, email and push notifications, role-based group permissions, approval workflows, payment-gateway integration, downloadable server-generated reports, and a mobile application. A production release should add migrations, automated browser tests, observability, backup restoration tests, and a documented data-retention policy.

The architecture can also be extended with a JSON API for mobile and third-party clients. The current route structure and database boundaries provide a reasonable foundation for that extension without changing the core domain model.

10. References

[1]: https://ieeexplore.ieee.org/document/720574 "IEEE Recommended Practice for Software Requirements Specifications"

[2]: https://flask.palletsprojects.com/ "Flask Documentation"

[3]: https://dev.mysql.com/doc/refman/8.0/en/ "MySQL 8.0 Reference Manual"

[4]: https://dev.mysql.com/doc/connector-python/en/ "MySQL Connector/Python Developer Guide"

[5]: https://werkzeug.palletsprojects.com/ "Werkzeug Documentation"

[6]: https://docs.authlib.org/ "Authlib Documentation"

[7]: https://mermaid.js.org/ "Mermaid Diagramming and Charting Documentation"

[8]: https://www.iso.org/standard/35733.html "ISO/IEC/IEEE 29148 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering"

[9]: https://www.coursera.org/articles/software-development-life-cycle-sdlc "Software Development Life Cycle Overview"

[10]: https://www.uml.org/ "Object Management Group UML Resource"

[11]: https://github.com/EvenUp-placeholder/project "EvenUp project source code and implementation record"